Data Structures

Lecture 9: Priority Queues

Prof. Wang Haiyan
School of Computer Science
wanghy@njupt.edu.cn

Today's Topic

We have been focussing on the Dictionary ADT

This supports insertion, retrieval and deletion of any element by its key

Today we will look at a similar but more restricted ADT

Rather than being able to retrieve any element from the structure, we can only remove the one with ***highest priority***

This is a ***priority queue***

Today's Topic

As for a dictionary, elements consist of a key/value pair

The key denotes the priority of the element

There is a total order over the keys

The element with the smallest (or sometimes largest) key is considered the highest priority

Priority Queues

A priority queue maintains a set of elements, M, on which the following operations can be performed:

$M.build(\{e_1, \ldots, e_n\}) : \quad M := \{e_1, \ldots, e_n\}.$

$M.insert(e) : \quad M := M \cup \{e\}.$

$M.findMin : \quad \textbf{return } \min(M).$

$M.deleteMin : \quad e := \min(M); M := M \setminus \{e\}; \textbf{return } e.$

Where “min” means “highest priority”

Q: Which of the data structures we’ve seen so far could be used to support these operations?

Priority Queues

A priority queue maintains a set of elements, M, on which the following operations can be performed:

$M.build(\{e_1, \ldots, e_n\})$: $M := \{e_1, \ldots, e_n\}$.
$M.insert(e)$: $M := M \cup \{e\}$.
$M.findMin$: **return** $\min(M)$.
$M.deleteMin$: $e := \min(M); M := M \setminus \{e\}$; **return** e.

Where “min” means “highest priority”

Q: Which of the data structures we’ve seen so far could be used to support these operations?

A: Any implementation of a dictionary could be used. Insertion will be the same. deleteMin requires us to find the minimum element in our dictionary and remove it.

Applications of Priority Queues

We unknowingly use priority queues in every day life:

“To do” lists: determine highest priority task, remove from list and carry it out. Priority may increase over time, or decrease if a deadline is moved back.

Shopping on a budget: we buy the most important items first until the total cost equals our budget, we will have to do without the remaining items in the queue.

Sending Christmas cards: choose highest priority person from queue, write them a card, repeat until we run out of cards/stamps. Remaining people on queue won’t get a card!

Many other examples

Applications of Priority Queues

Although seemingly an inferior version of a dictionary, priority queues can be used as part of algorithms to solve many important problems:

Sorting: We can use a priority queue for a sort of selection sort. We first insert all our elements into the priority queue, then repeatedly delete the minimum. This sequence of minima will give us the sorted sequence.

Resource usage management: Processor time, disc access, communication bandwidth etc are all shared by multiple processes, data streams etc. Priority queues can be used to decide what should next get access to the resource. Priority can increase with time so that items don't get stuck in queue forever.

Event queues: We can define priority as the time at which an event happens. To simulate a sequence of discrete events, we just continually remove the next event from the priority queue and update state accordingly.

Notions of “Priority”

What we mean by “priority” is entirely application dependent

Examples

Highest priority equals…

1. … the most recently inserted item
2. … the item which has been waiting longest
3. … the event which will happen next

Many other possibilities

Sometimes, what we mean by priority will effect the optimal implementation of the priority queue

Q: Can you suggest a good way to implement a priority queue for cases 1 and 2 above?

Notions of "Priority"

What we mean by "priority" is entirely application dependent

Examples

Highest priority equals…

1. … the most recently inserted item
2. … the item which has been waiting longest
3. … the event which will happen next

Many other possibilities

Sometimes, what we mean by priority will effect the optimal implementation of the priority queue

Q: Can you suggest a good way to implement a priority queue for cases 1 and 2 above?

A: A stack and a queue

Priority Queues as Stacks

If highest priority means most recently inserted, use a stack

Insert = push

DeleteMin = pop

Q: Efficiency of priority queue operations using a stack implementation?

Priority Queues as Stacks

If highest priority means most recently inserted, use a stack

Insert = push

DeleteMin = pop

Q: Efficiency of priority queue operations using a stack implementation?

A: O(1), with stack implemented as array or linked list

Priority Queues as Queues

If highest priority means the element that has been waiting longest, use a queue

Insert = EnQueue

DeleteMin = DeQueue

Q: Efficiency of priority queue operations using a queue implementation?

Priority Queues as Queues

If highest priority means the element that has been waiting longest, use a queue

Insert = EnQueue

DeleteMin = DeQueue

Q: Efficiency of priority queue operations using a queue implementation?

A: O(1), with queue implemented as array or linked list

Conclusion: if your application uses a notion of priority which is either the most recently added or longest-waiting element, a queue- or stack-based implementation will be optimal.

For the general case we need other structures…

Naïve Priority Queue Implementations

We've said that a priority queue is like a restricted version of a dictionary

How will the dictionary implementations we've seen so far perform for a priority queue?

Options:

- Sorted/unsorted array
- Sorted/unsorted linked list
- Binary search tree
- Hash table

Naïve Priority Queue Implementations

Efficiency of priority queue operations using a dictionary implemented as an array (standard insert and find operations):

Operation	Dictionary implemented as unsorted array	Dictionary implemented as sorted array
`M.insert(e)`	O(1)	O(n)
`M.findMin`	O(n)	O(1)
`M.deleteMin`	O(n)	O(n)

Q: Can we do better using arrays?

Naïve Priority Queue Implementations

Efficiency of priority queue operations using a dictionary implemented as an array (standard insert and find operations):

Operation	Dictionary implemented as unsorted array	Dictionary implemented as sorted array
`M.insert(e)`	O(1)	O(n)
`M.findMin`	O(n)	O(1)
`M.deleteMin`	O(n)	O(n)

Q: Can we do better using arrays?

A: Priority queues are only interested in the minimum item, we don't need the full power of the generic dictionary delete operation (we'll only ever delete min). We can also store the index of the current minimum for an unsorted array.

Naïve Priority Queue Implementations

Efficiency of priority queue operations using better array-based implementations:

Operation	Unsorted array	Sorted array
`M.insert(e)`	O(1)	O(n)
`M.findMin`	O(1)	O(1)
`M.deleteMin`	O(n)	O(1)

To achieve this we just store the index of the item with the current minimum value

Q: What impact will this have on insert and deleteMin?

Naïve Priority Queue Implementations

Efficiency of priority queue operations using better array-based implementations:

Operation	Unsorted array	Sorted array
`M.insert(e)`	O(1)	O(n)
`M.findMin`	O(1)	O(1)
`M.deleteMin`	O(n)	O(1)

To achieve this we just store the index of the item with the current minimum value

Q: What impact will this have on insert and deleteMin?

A: Insertion remains O(1), just compare new item to current minimum and update min index if necessary. DeleteMin stays O(n), we'll have to search for the new minimum when we delete the current one.

Naïve Priority Queue Implementations

Efficiency of priority queue operations using better array-based implementations:

Operation	Unsorted array	Sorted array
`M.insert(e)`	O(1)	O(n)
`M.findMin`	O(1)	O(1)
`M.deleteMin`	O(n)	O(1)

Q: How do we achieve this?

Naïve Priority Queue Implementations

Efficiency of priority queue operations using better array-based implementations:

Operation	Unsorted array	Sorted array
`M.insert(e)`	O(1)	O(n)
`M.findMin`	O(1)	O(1)
`M.deleteMin`	O(n)	O(1)

Q: How do we achieve this?

A: Store the list in reverse sorted order, so minimum element is last in array. To remove it, just reduce index to last element in priority queue by 1 – no shuffling needed!

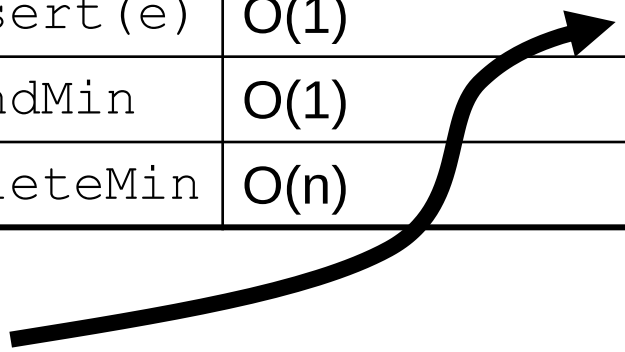
Similar logic can be used for sorted/unsorted linked list

Performance with a sorted array doesn't look too bad…

Naïve Priority Queue Implementations

Efficiency of priority queue operations using better array-based implementations:

Operation	Unsorted array	Sorted array
`M.insert(e)`	O(1)	O(n)
`M.findMin`	O(1)	O(1)
`M.deleteMin`	O(n)	O(1)



This is the problem

Linear time insert is going to be really bad for priority queues containing a large number of items

Naïve Priority Queue Implementations

Q: Are hash tables a good choice for implementing priority queues?

Naïve Priority Queue Implementations

Q: Are hash tables a good choice for implementing priority queues?

A: No!

Remember, we intentionally wanted our hash function to distribute keys at random through the array

We have no way to find the minimum value beyond a linear scan through the array, much of which may be empty

We can keep a reference to the current minimum as for an unsorted array

So findMin is O(1)

But DeleteMin is O(n) – mark old minimum as a ghost, linear search to find new minimum

Insert will be O(1) on average, but could degenerate to O(n)

In practice, worse than an unsorted array

Naïve Priority Queue Implementations

Efficiency of priority queue operations using a dictionary implemented as a **balanced binary search tree**:

Insert: O(log n)

Recall: O(log n) to find correct place to insert plus O(log n) for potential rebalancing

Q: Revision: complexity of **find minimum**?

Naïve Priority Queue Implementations

Efficiency of priority queue operations using a dictionary implemented as a **balanced binary search tree**:

Insert: O(log n)

Recall: O(log n) to find correct place to insert plus O(log n) for potential rebalancing

Q: Revision: complexity of **find minimum**?

A: We continually follow left children giving O(log n)

Q: How about **delete minimum**?

Naïve Priority Queue Implementations

Efficiency of priority queue operations using a dictionary implemented as a **balanced binary search tree**:

Insert: O(log n)

Recall: O(log n) to find correct place to insert plus O(log n) for potential rebalancing

Q: Revision: complexity of **find minimum**?

A: We continually follow left children giving O(log n)

Q: How about **delete minimum**?

A: We find it in O(log n) time, delete in O(1), then rebalance in O(log n), total complexity: O(log n)

So all the priority queue operations are O(log n) with a binary tree

However, we don’t need the full power of a binary tree…

Binary Heaps

A balanced binary search tree implementation allows retrieval of **any element** by key in O(log n) time

But our priority queue only needs to find the **minimum element**

With a less flexible structure, can we make operations involving the minimum element more efficient?

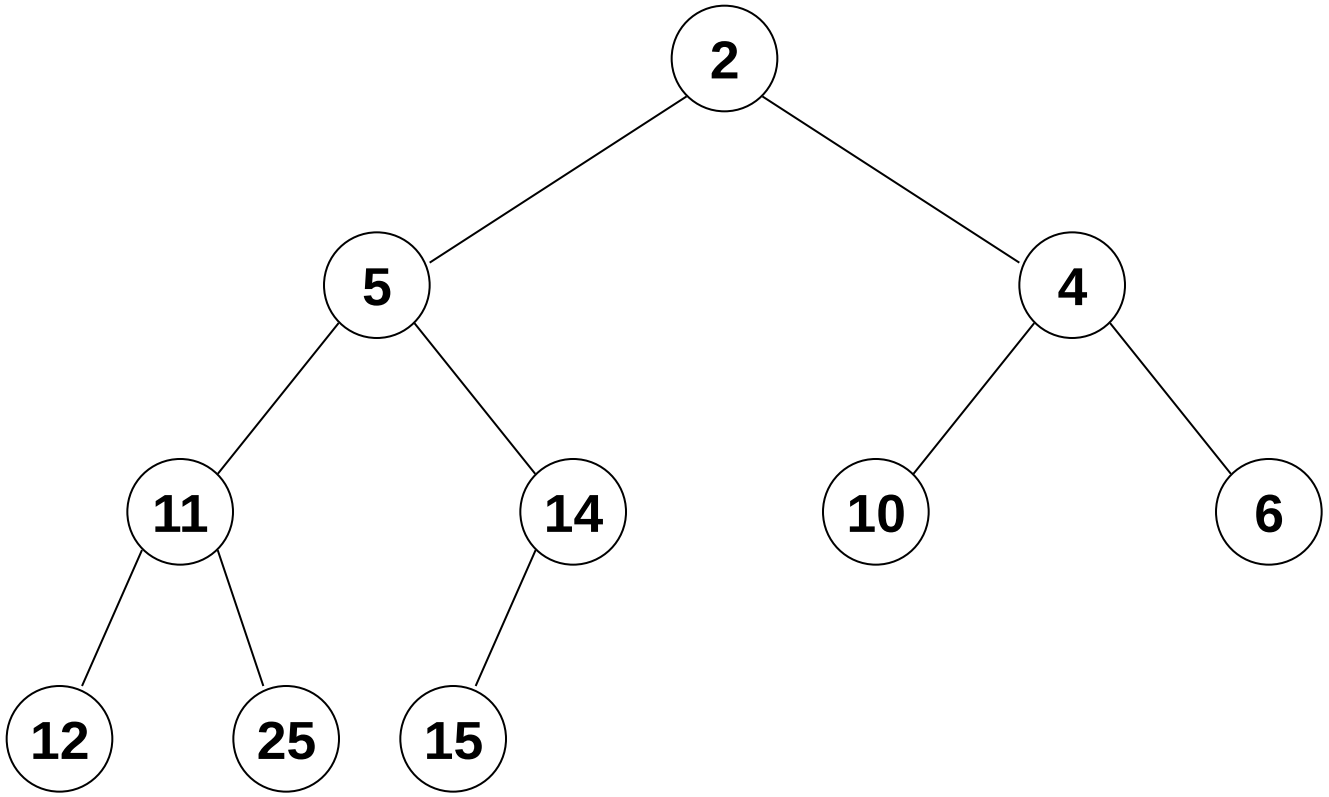
A useful structure for doing this is a ***binary heap***

A binary heap is a **complete** binary tree (all levels full, apart from the lowest level which is filled from left to right)

A binary heap has a weaker invariant than a binary search tree:

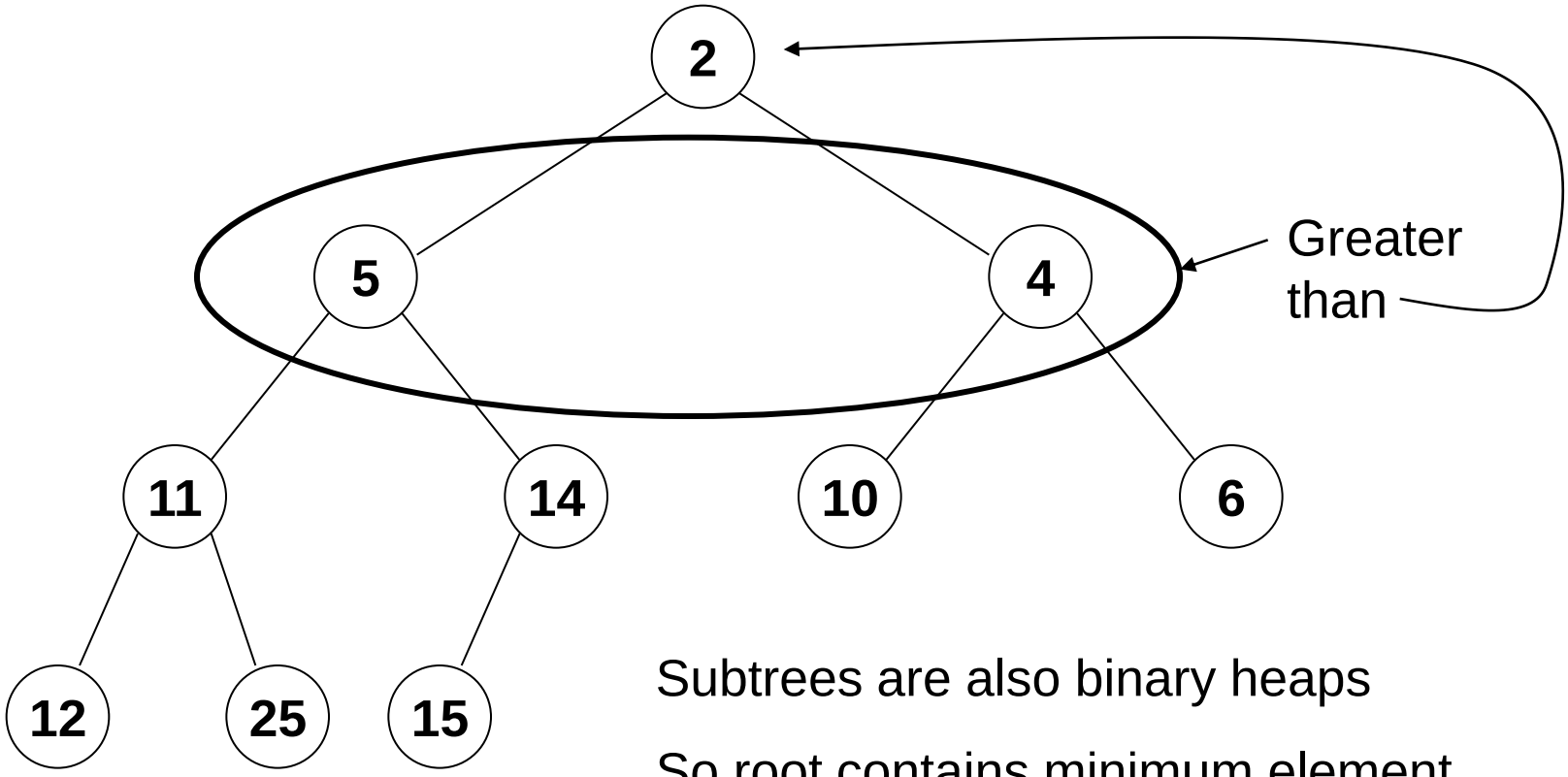
No child has a key less than its parent's

Binary Heaps



A heap can be classified further as either a "**max heap**" or a "**min heap**".

Binary Heaps



Subtrees are also binary heaps

So root contains minimum element

Binary Heaps

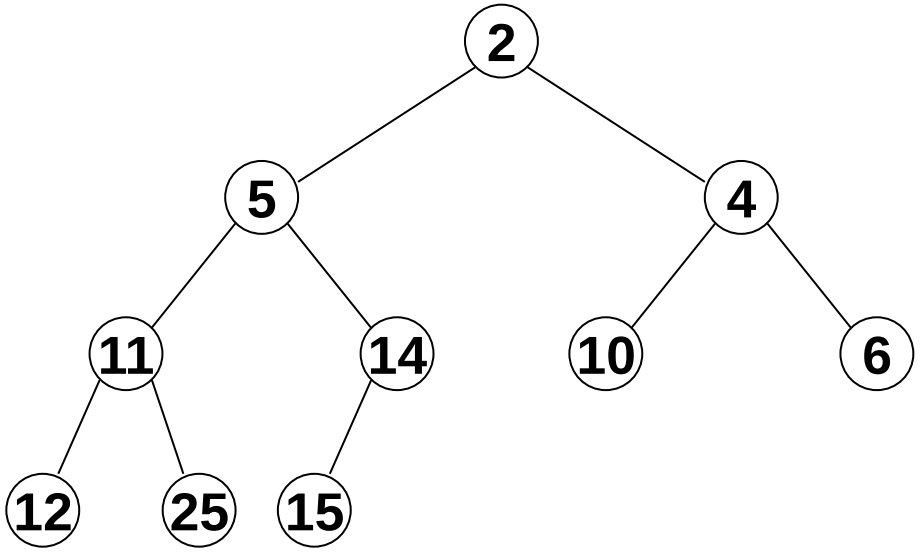
We will begin by considering how the priority queue operations are implemented using a binary heap

We will then see how a binary heap is implemented using concrete data structures

Priority Queues using Binary Heaps

`M.findMin`

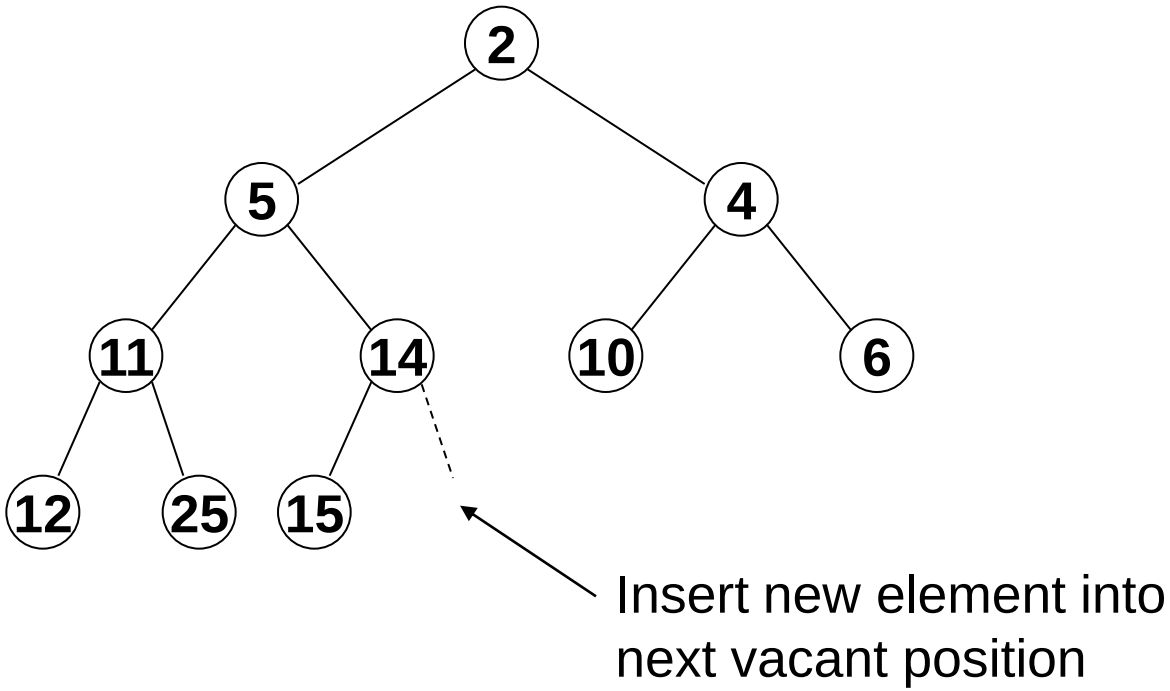
Since the root contains the minimum value, we simply return the root:



Priority Queues using Binary Heaps

`M.insert(e)`

The next vacant position is the leftmost on the bottom row, or if the bottom row is full, the left child of the left most element on the bottom row

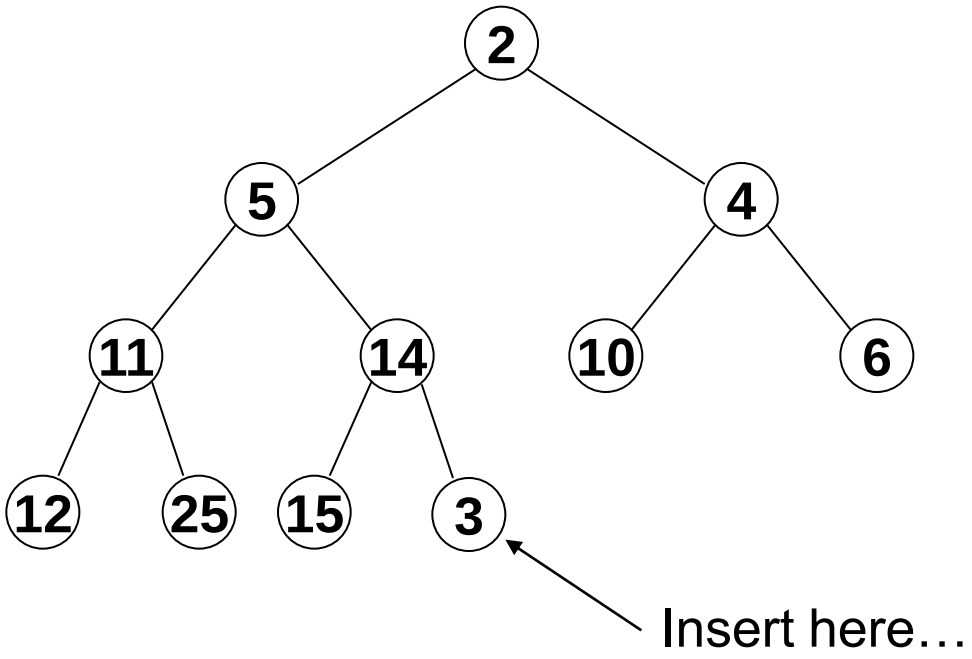


Q: How can we ensure the heap invariant is restored?

Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

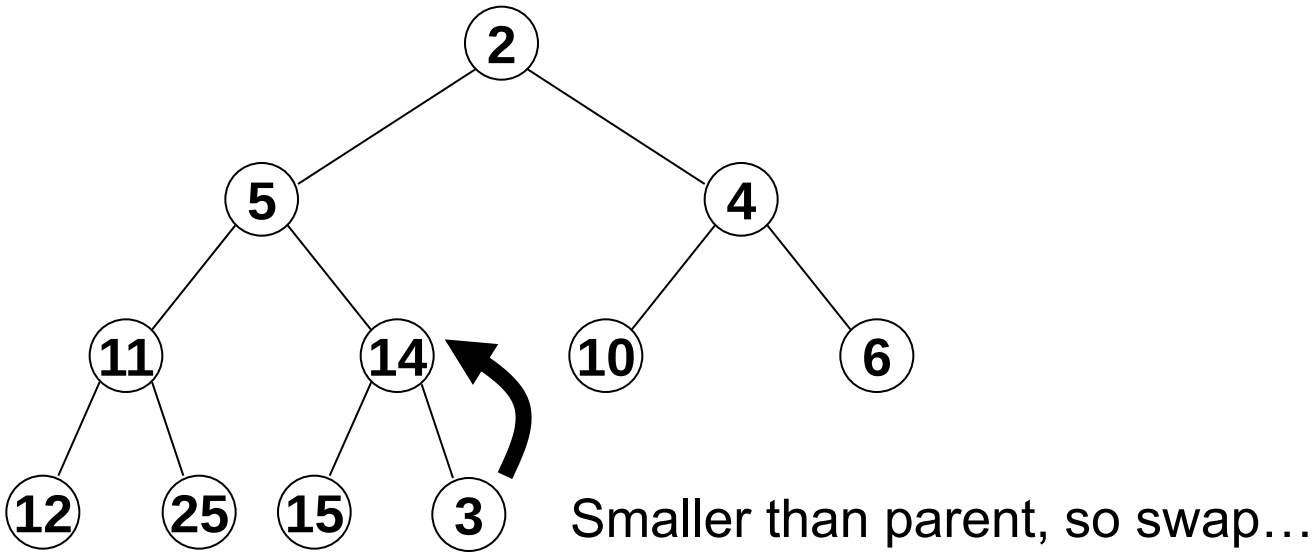
E.g. insert 3:



Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

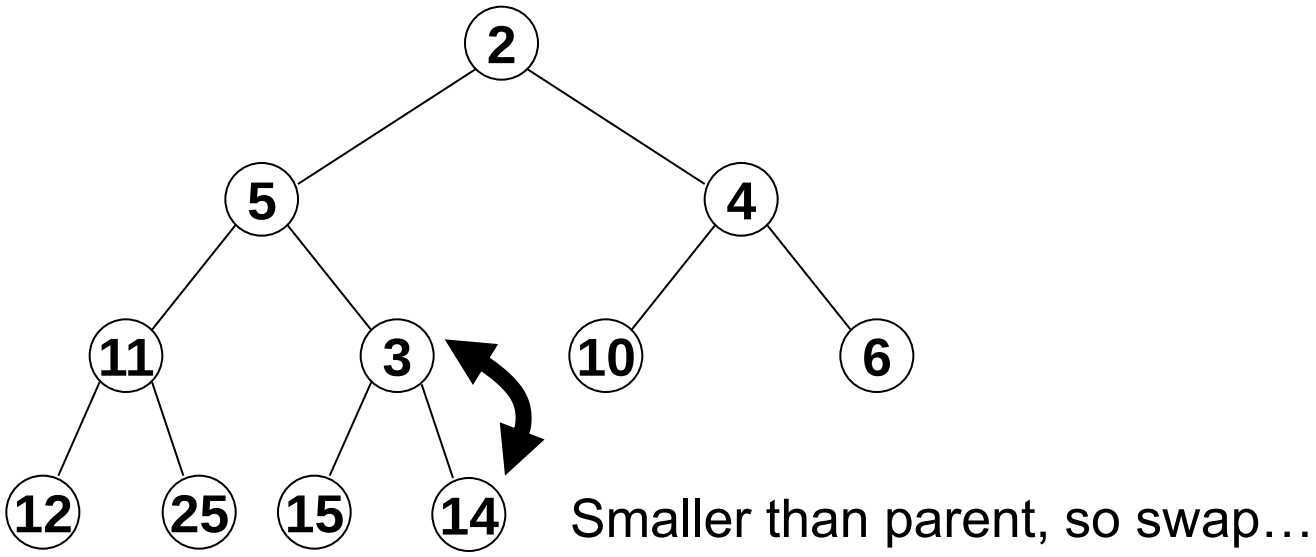
E.g. insert 3:



Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

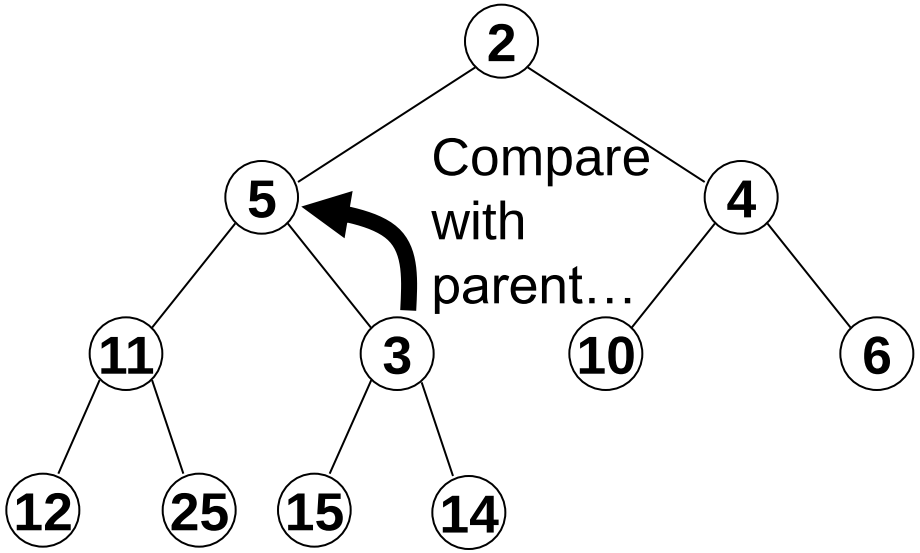
E.g. insert 3:



Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

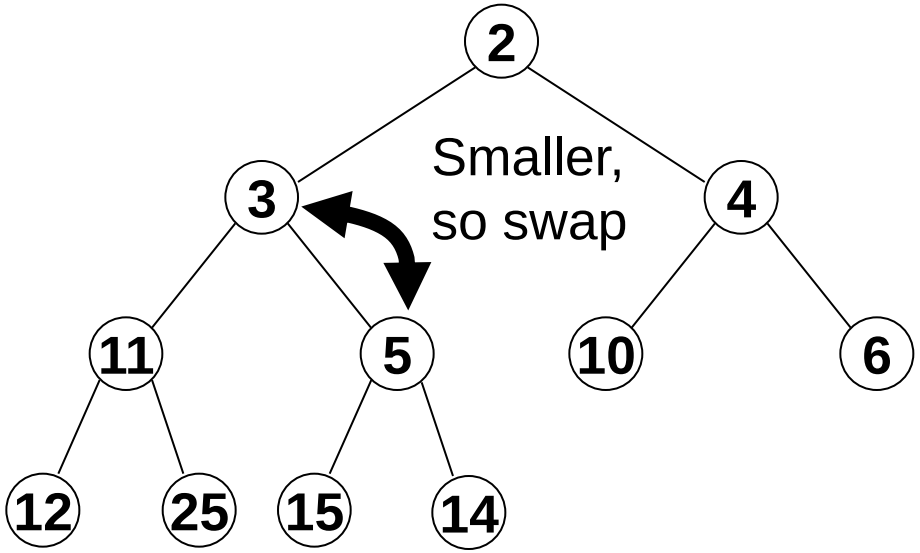
E.g. insert 3:



Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

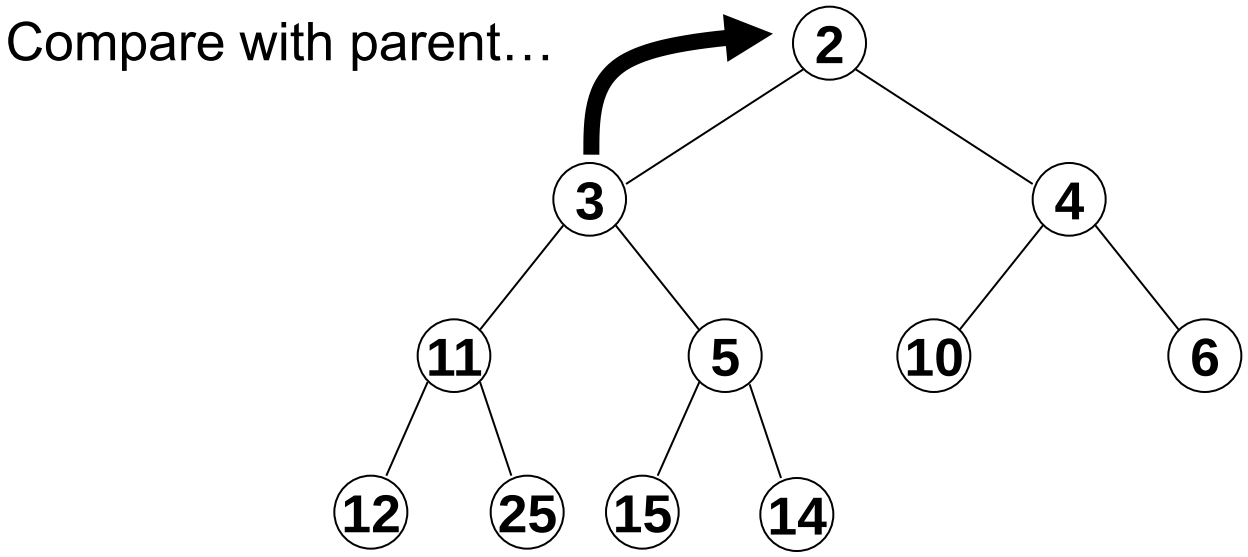
E.g. insert 3:



Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

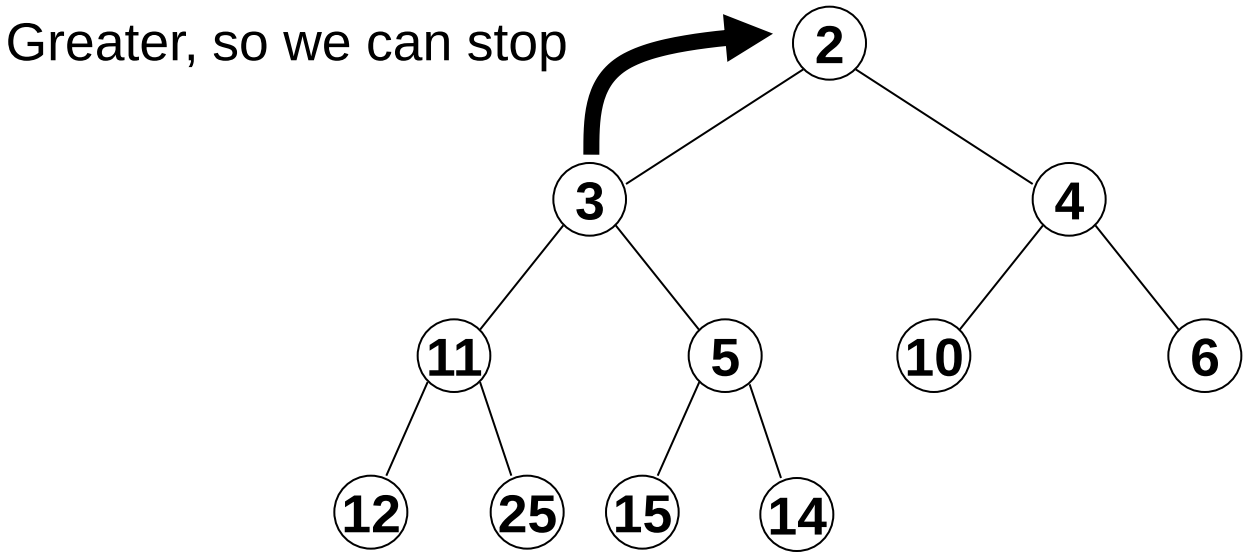
E.g. insert 3:



Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

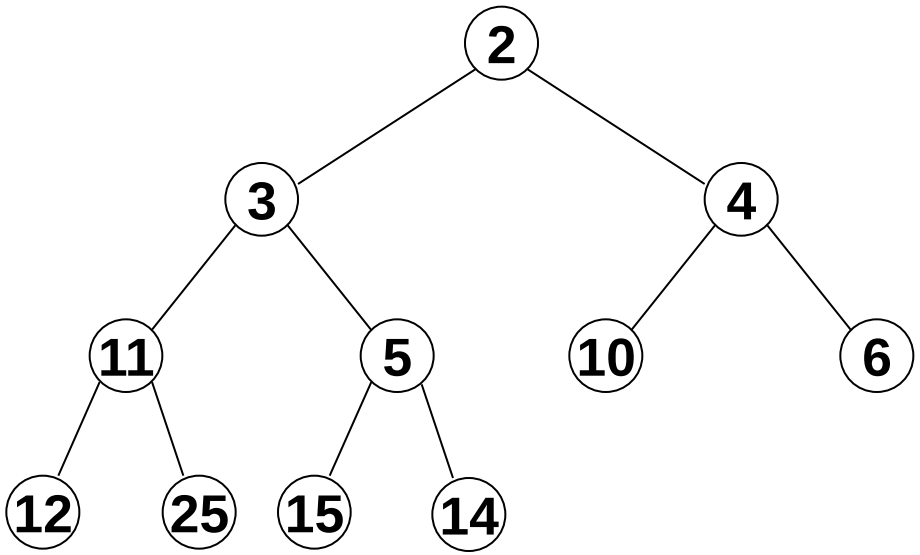
E.g. insert 3:



Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

E.g. insert 3:

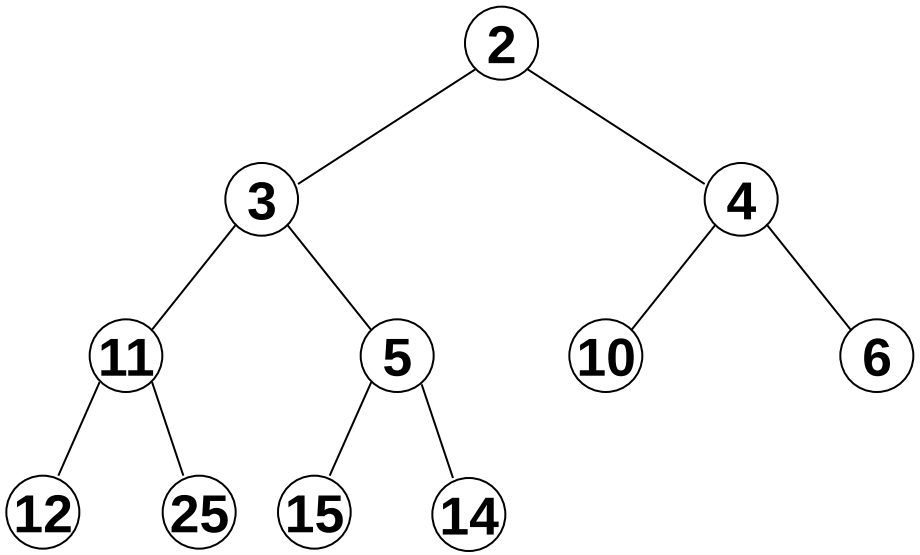


Q: Worst case complexity for insertion?

Priority Queues using Binary Heaps

A: “Bubble up” new element until it has a parent with a lower value

E.g. insert 3:



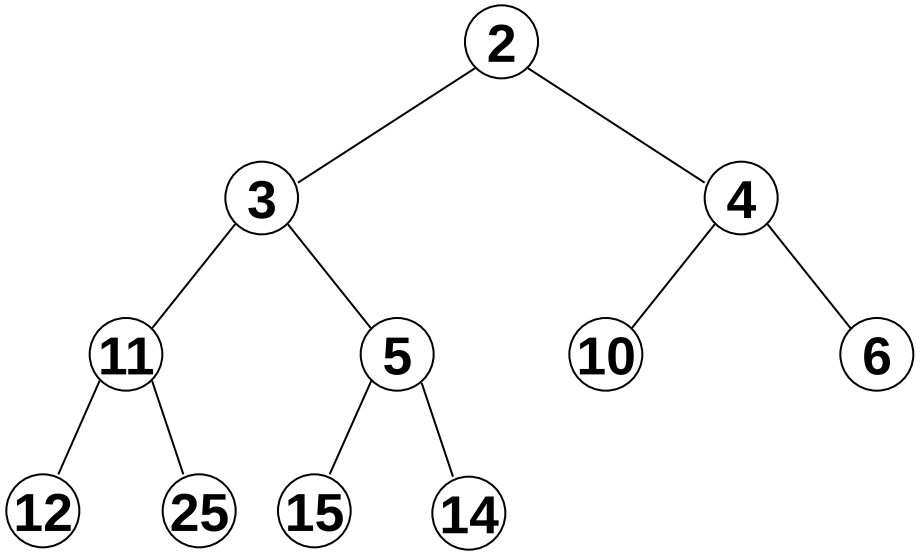
Q: Worst case complexity for insertion?

A: For n elements, tree depth is O(log n), so worst case complexity is O(log n)

Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

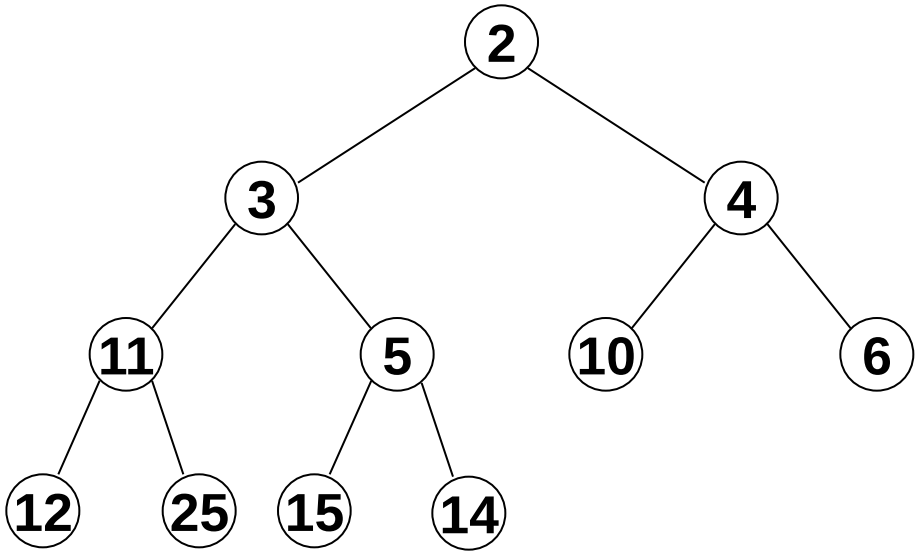


Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…

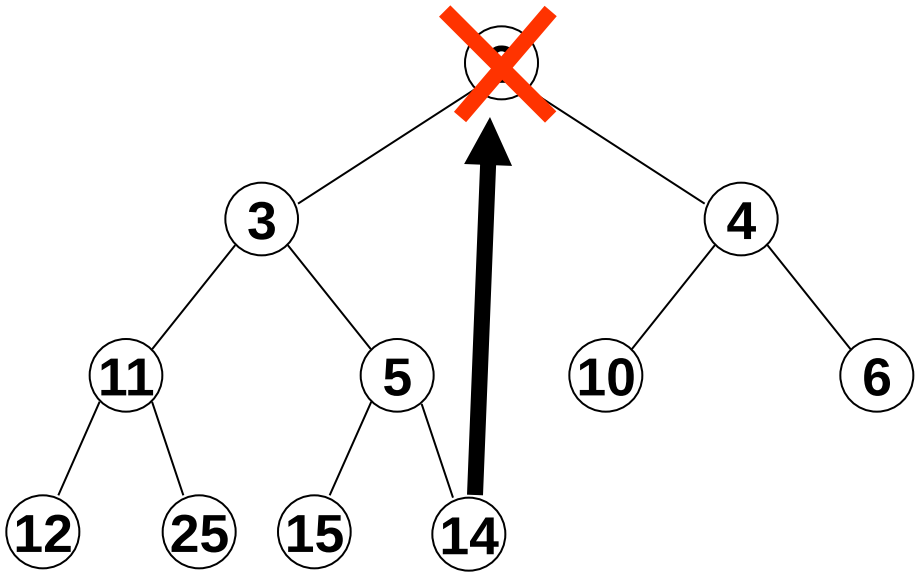


Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…

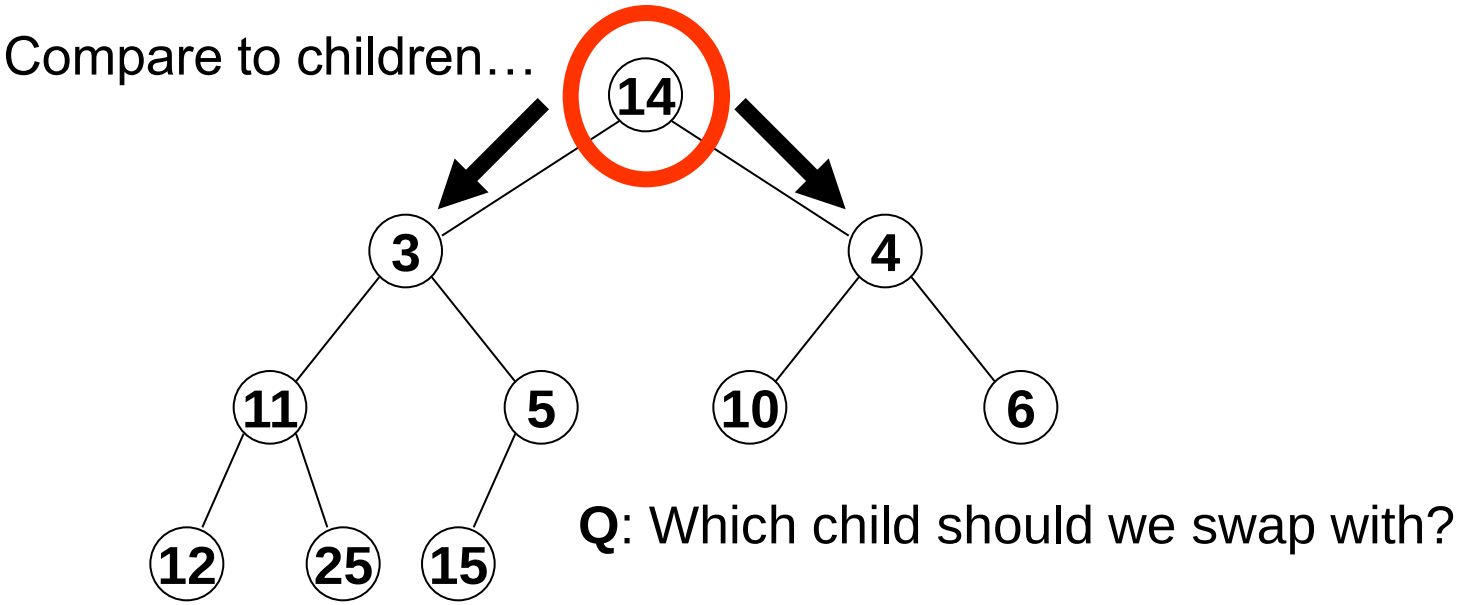


Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…

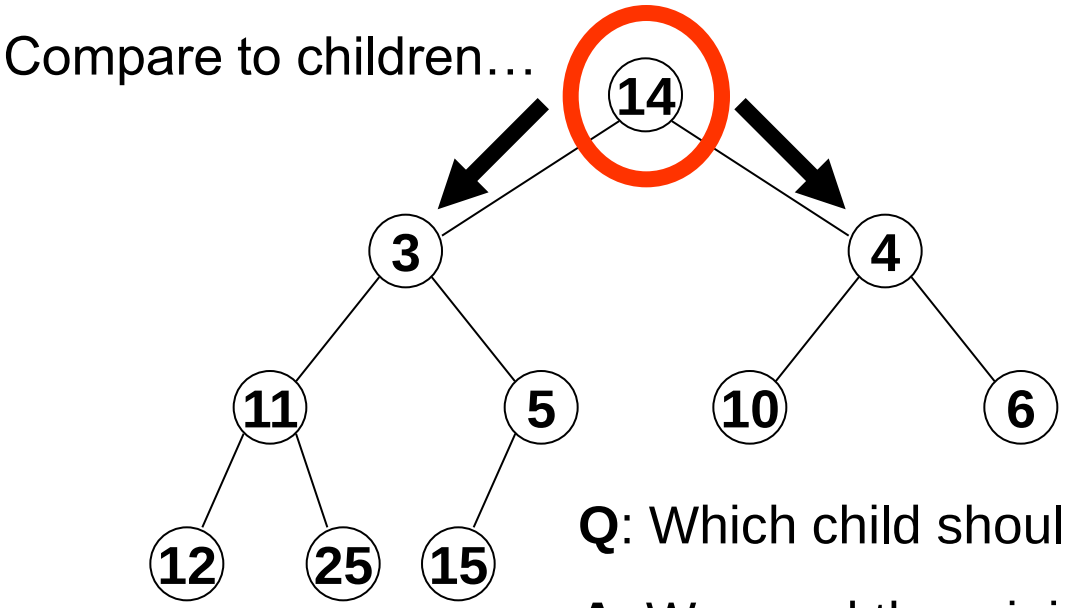


Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…



Q: Which child should we swap with?

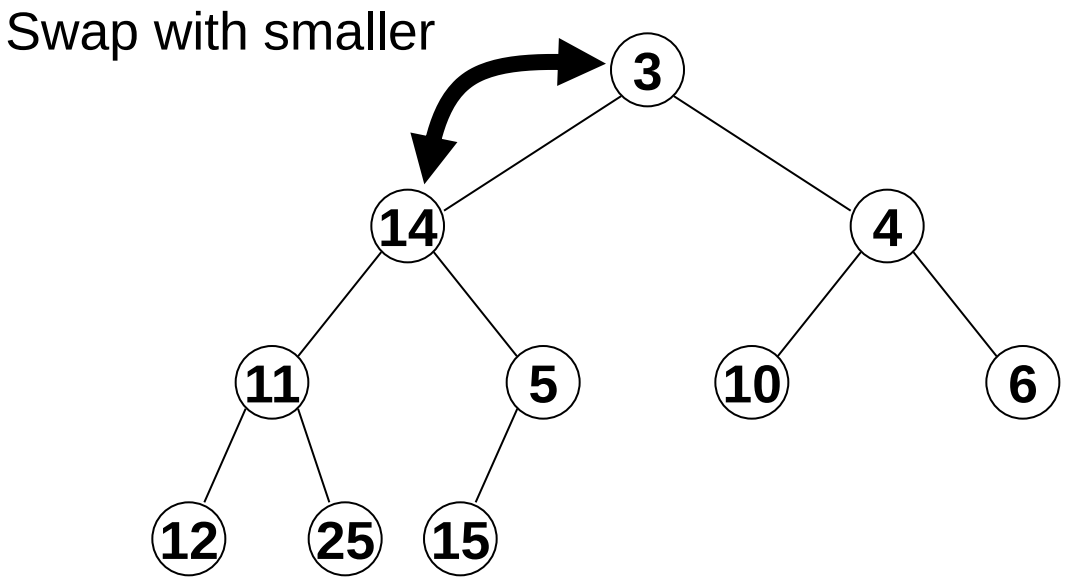
A: We need the minimum value at the root so we swap with the smaller of the two children

Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…

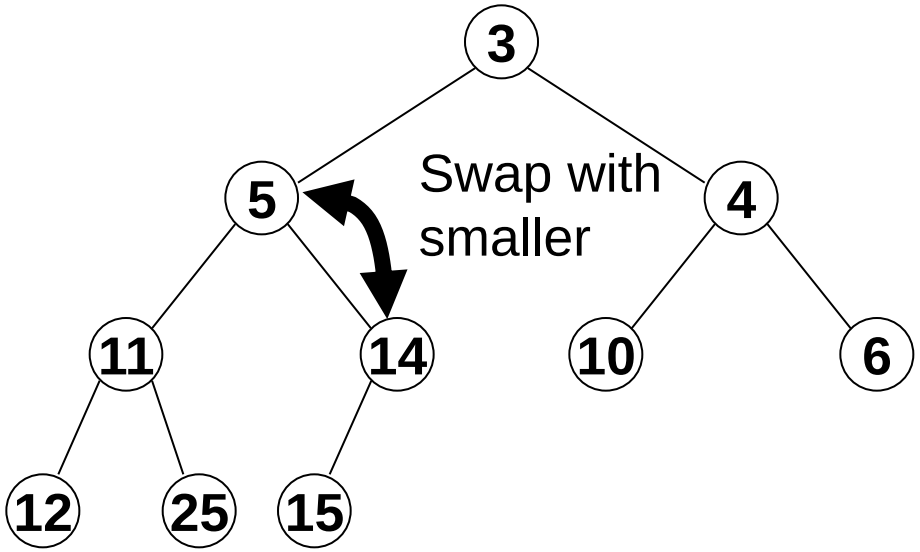


Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…

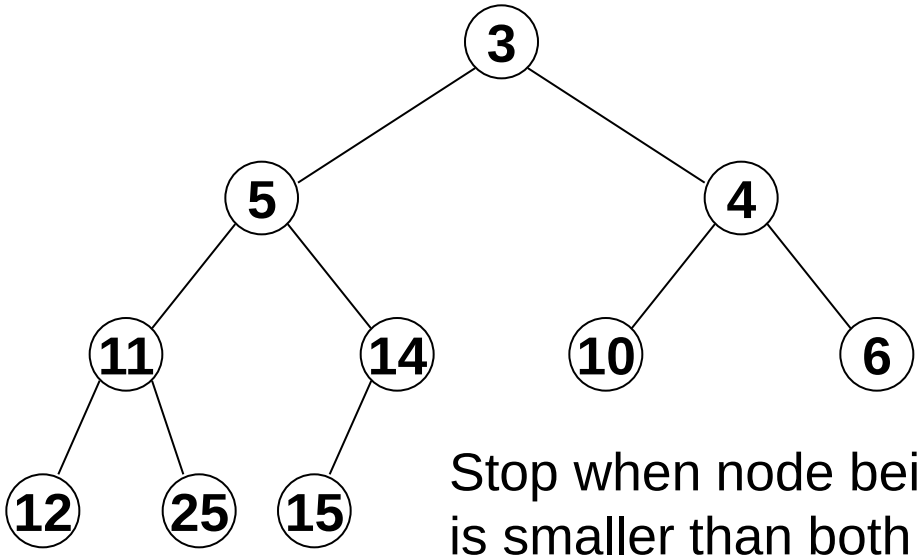


Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…



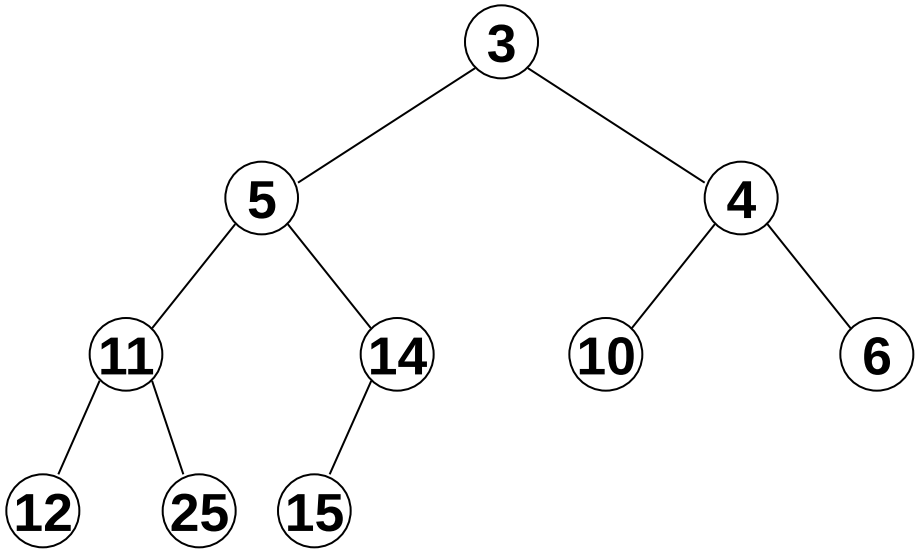
Stop when node being bubbled down is smaller than both its children

Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…



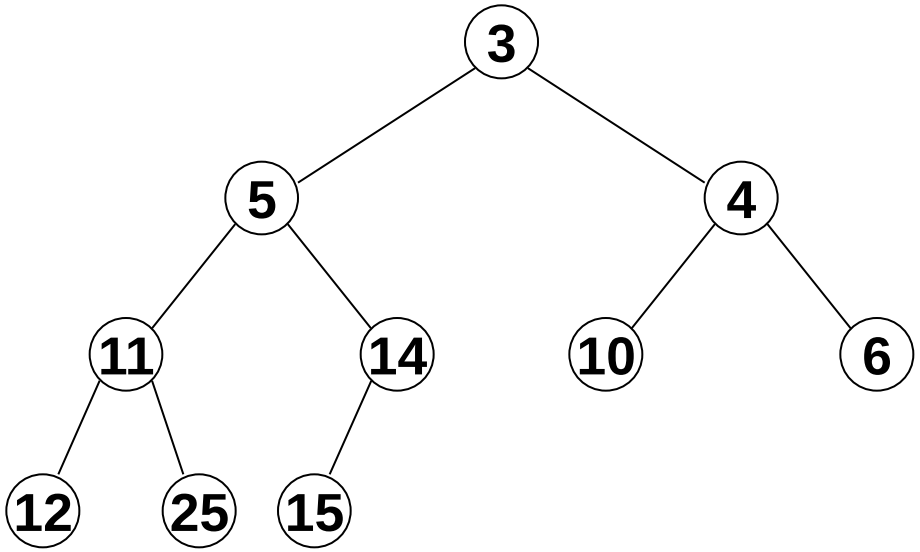
Q: Worst case complexity for deleting root?

Priority Queues using Binary Heaps

DeleteMin requires removing the root node from the tree

Q: How might we delete the root while maintaining the heap invariant?

A: Replace it with the last element and bubble this down…



Q: Worst case complexity for deleting root?

A: We must bubble all the way to the bottom: O(log n)

Priority Queues using Binary Heaps

So using a binary heap, we can find the minimum in O(1) time and can remove the minimum or insert in O(log n) time

Summary:

Operation	Unsorted array	Sorted array	Binary heap
`M.insert(e)`	O(1)	O(n)	O(log n)
`M.findMin`	O(1)	O(1)	O(1)
`M.deleteMin`	O(n)	O(1)	O(log n)

Implementing Binary Heaps

We can store a binary tree using either an array or linked structure

Q: Why didn't we use an array for a balanced binary search tree?

Implementing Binary Heaps

We can store a binary tree using either an array or linked structure

Q: Why didn't we use an array for a balanced binary search tree?

A: Lack of flexibility – rotation operations would become hugely messy

But binary heaps don't need rotations, bubbling up and down only requires swapping elements…

Binary Heaps as Arrays

Unusually for a tree structure, the most efficient concrete implementation for a binary heap is an array

Recall: storing trees in arrays from lecture 7

Root is stored in array slot 1

Followed by each level of the tree in left to right order

Children of node i are in slots 2i and 2i+1

Parent of node i is in: $\left\lfloor \frac{i}{2} \right\rfloor$

Binary Heaps as Arrays

Unusually for a tree structure, the most efficient concrete implementation for a binary heap is an array

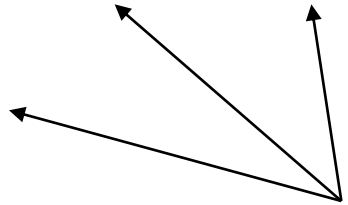
Recall: storing trees in arrays from lecture 7

Root is stored in array slot 1

Followed by each level of the tree in left to right order

Children of node i are in slots 2i and 2i+1

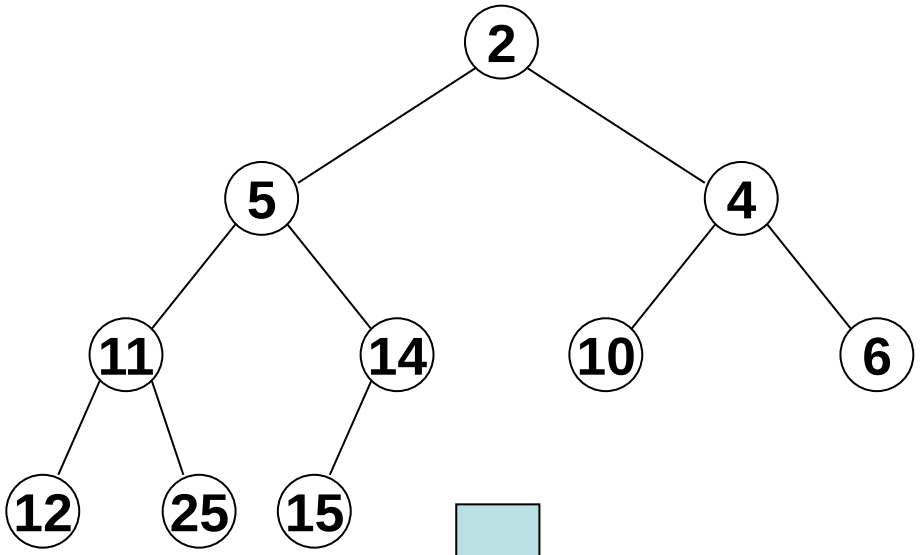
Parent of node i is in: $\left\lfloor \frac{i}{2} \right\rfloor$



Even better, we can do these operations extremely quickly using bit shifting, so navigating a binary tree in an array is very efficient

Binary Heaps as Arrays

Example:



Because the tree is complete, no slots are empty so no wasted space in array

1	2	3	4	5	6	7	8	9	10
2	**5**	**4**	**11**	**14**	**10**	**6**	**12**	**25**	**15**

Binary Heaps as Arrays

We keep a reference to the next free slot, e.g. 11:

1	2	3	4	5	6	7	8	9	10	11
2	**5**	**4**	**11**	**14**	**10**	**6**	**12**	**25**	**15**	**…**

This is where the next insertion will take place (and corresponds to left most free slot on lowest level)

When bubbling up, we compare element with parent, computed from: $\left\lfloor \frac{i}{2} \right\rfloor$

Bubbling up and bubbling down operations will be very fast (just swapping array elements and index calculations are bit shifts)

Changing Priorities

Q: Suppose you are given the array index position of a node and are told to change its priority to a given value, what would we need to do?

Changing Priorities

Q: Suppose you are given the array index position of a node and are told to change its priority to a given value, what would we need to do?

A: After changing its priority value (wherever that may be stored), the heap invariant may be broken

Changing Priorities

Q: Suppose you are given the array index position of a node and are told to change its priority to a given value, what would we need to do?

A: After changing its priority value (wherever that may be stored), the heap invariant may be broken

We would need to check whether it has become smaller than its parent or greater than its children and bubble up/down

Often, priority queues support the operation IncreasePriority, this allows us to increase an elements priority over time

In this case we just store the new priority and only need to check against parent and potentially bubble up

Binary Heaps as Arrays

Q: You are given an array containing n elements, what is the complexity of constructing a binary heap of these elements?

Binary Heaps as Arrays

Q: You are given an array containing n elements, what is the complexity of constructing a binary heap of these elements?

A: O(n log n), since each insertion takes O(log n) and we must do this n times

However, it turns out we can do better

Binary Heaps as Arrays

Q: You are given an array containing n elements, what is the complexity of constructing a binary heap of these elements?

A: O(n log n), since each insertion takes O(log n) and we must do this n times

However, it turns out we can do better

Even better, we can build a heap **in place**, i.e. we don't need any additional storage

Intuition: If we take an unsorted array, we can consider this a binary heap – the shape will be right, but the heap invariant will be broken everywhere

However, all leaf nodes will be valid (but very small) binary heaps

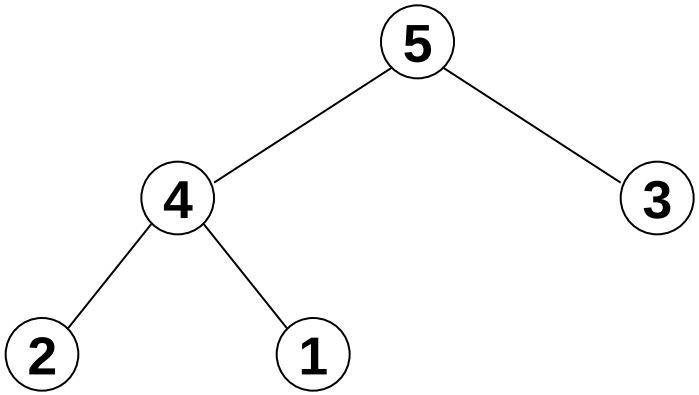
We can work our way backwards through the non-leaf nodes, fixing the small subheaps using the bubble down strategy

Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

1	2	3	4	5
5	**4**	**3**	**2**	**1**

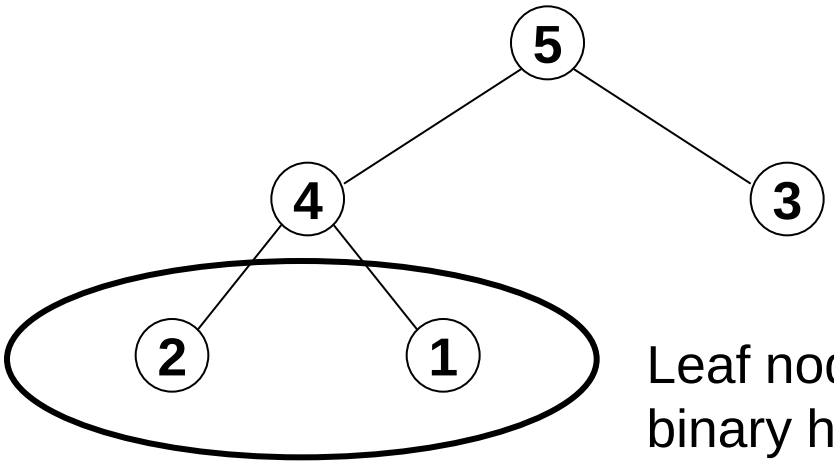


Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

1	2	3	4	5
5	**4**	**3**	**2**	**1**

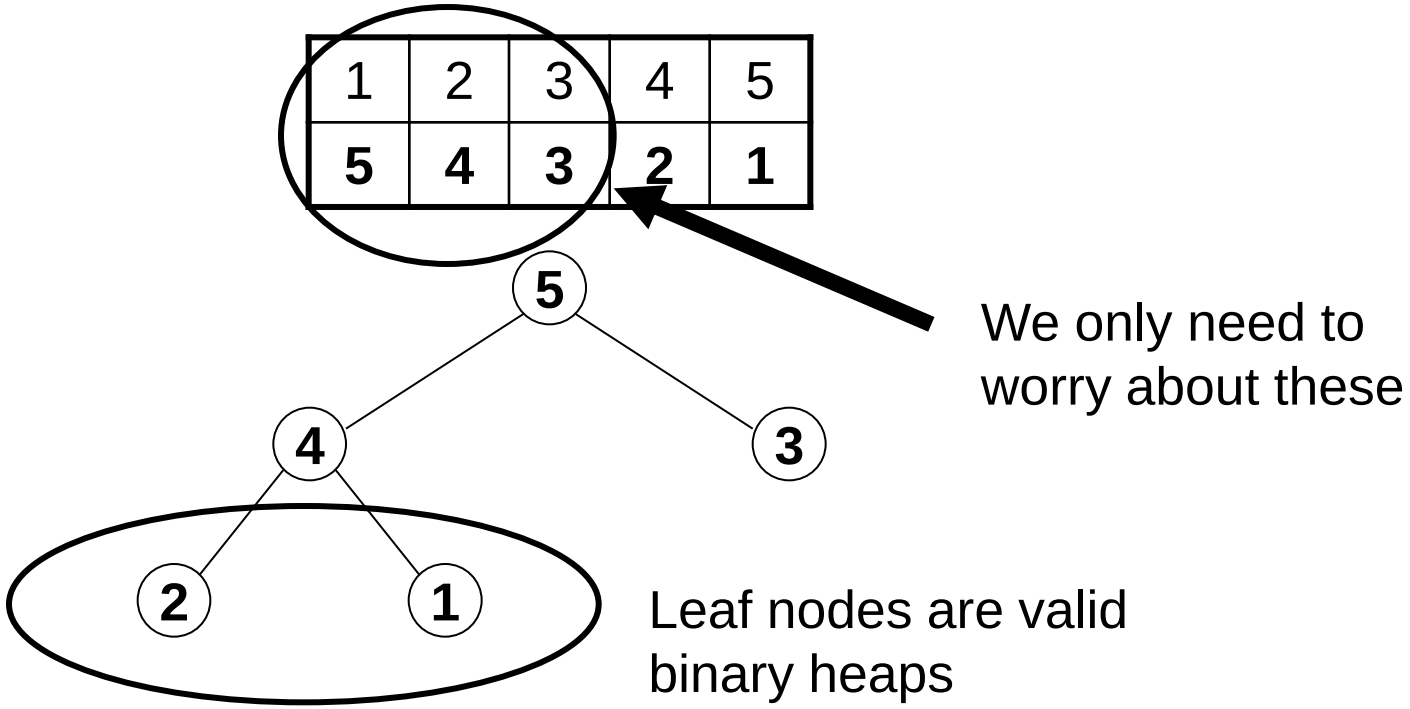


Leaf nodes are valid binary heaps

Binary Heaps as Arrays

Example

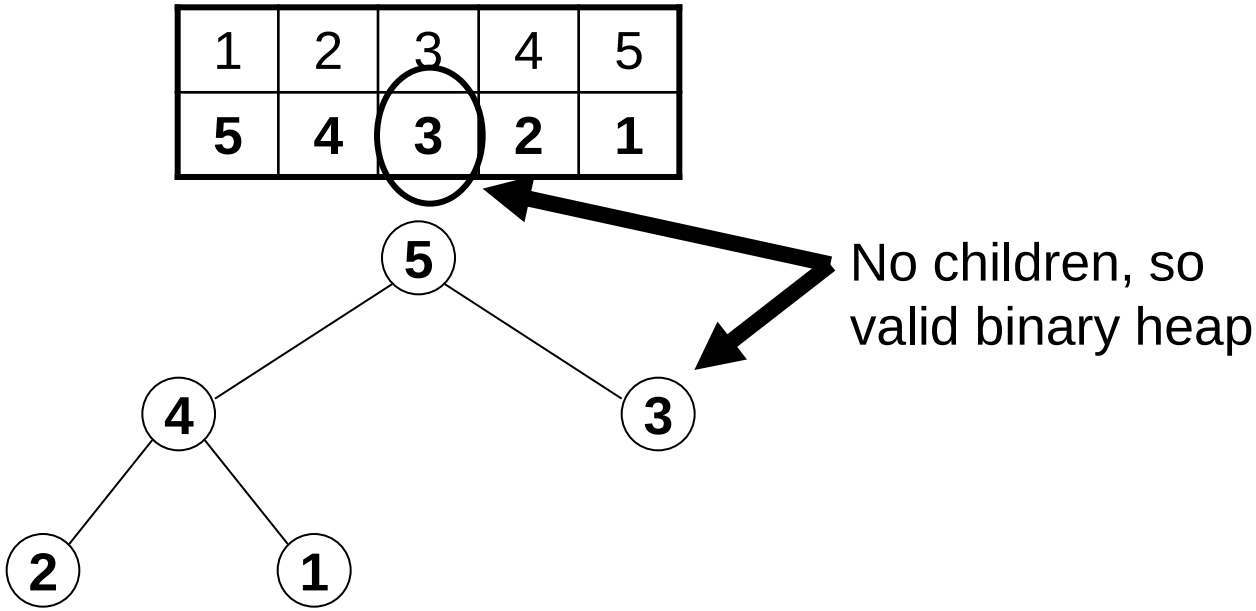
Let's take this unsorted array and consider it a binary heap:



Binary Heaps as Arrays

Example

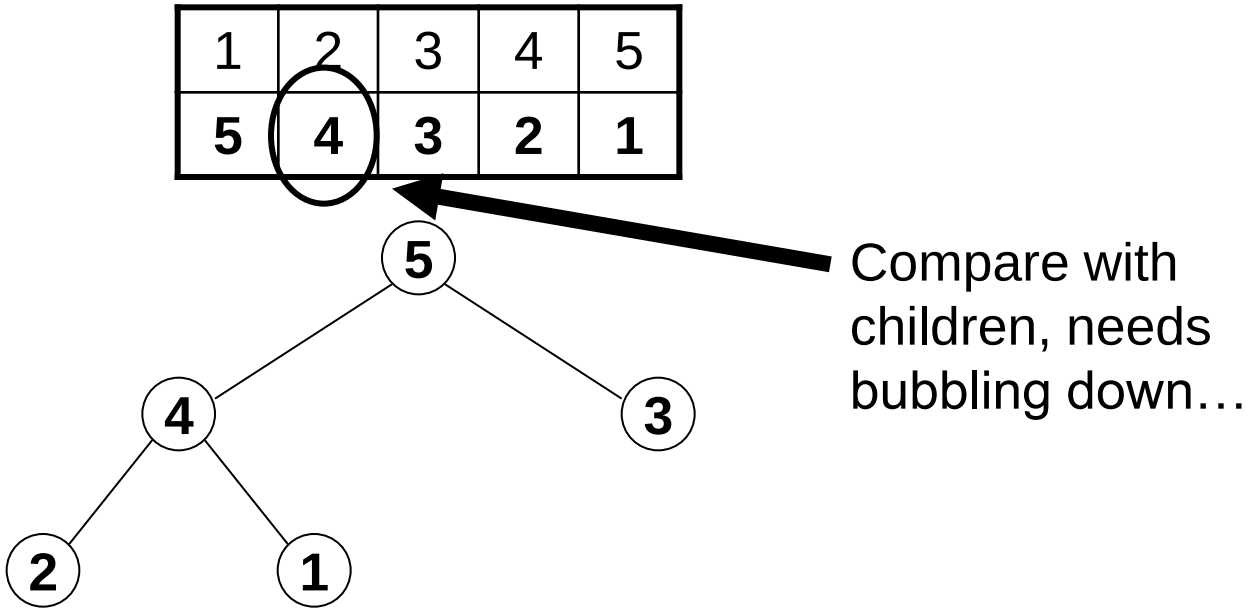
Let's take this unsorted array and consider it a binary heap:



Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

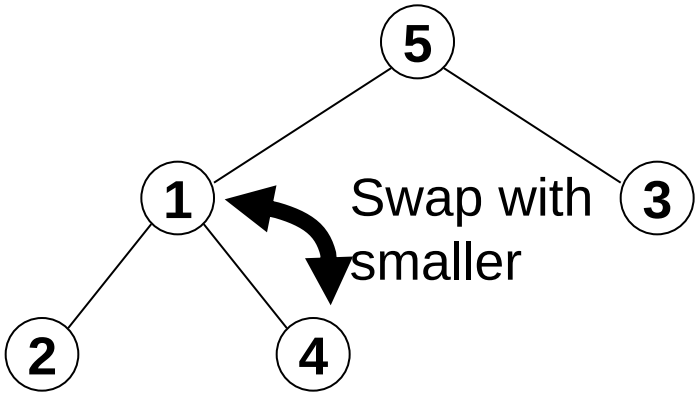


Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

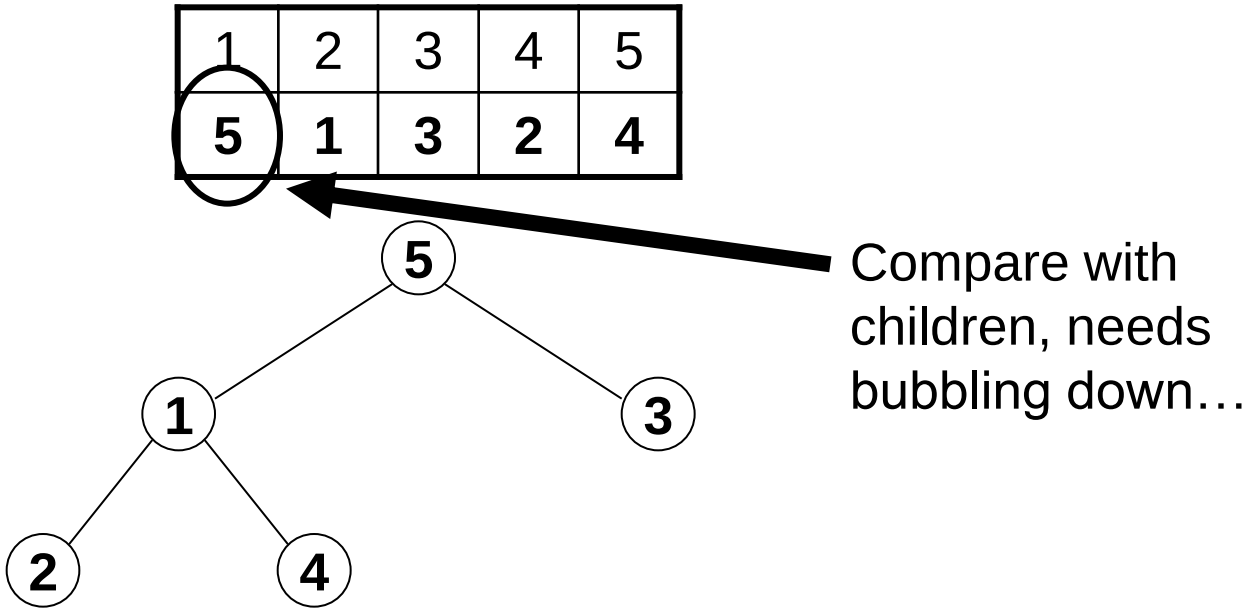
1	2	3	4	5
5	**1**	**3**	**2**	**4**



Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

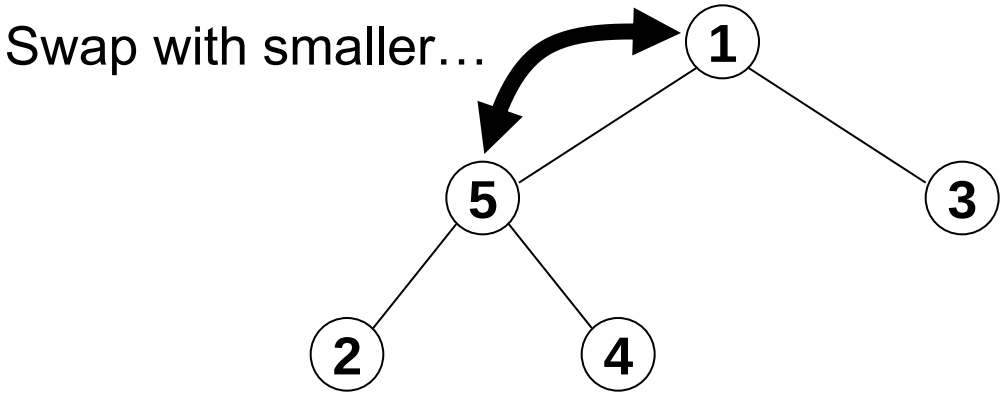


Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

1	2	3	4	5
1	**5**	**3**	**2**	**4**



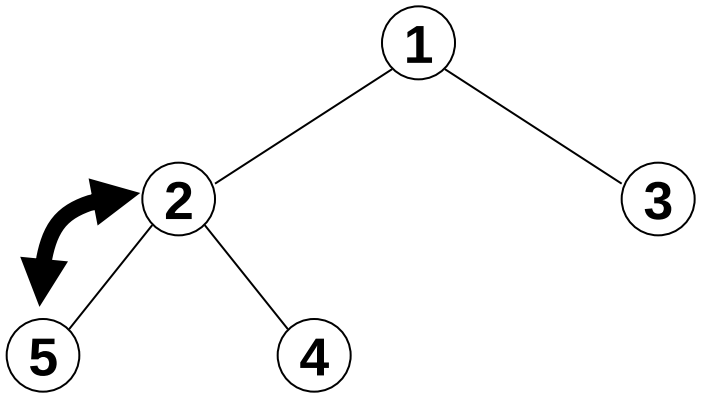
Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

1	2	3	4	5
1	**2**	**3**	**5**	**4**

Swap with smaller…

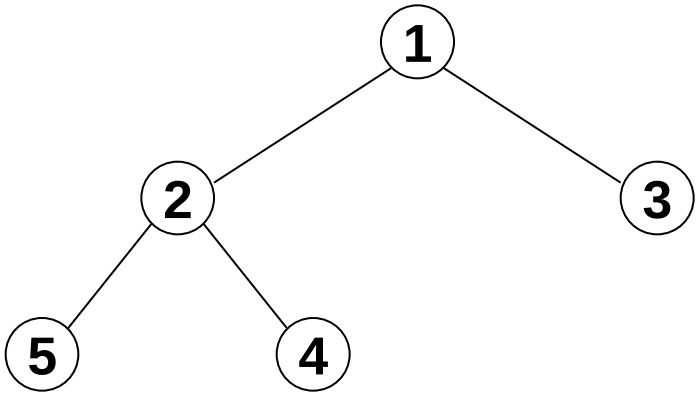


Binary Heaps as Arrays

Example

Let's take this unsorted array and consider it a binary heap:

1	2	3	4	5
1	**2**	**3**	**5**	**4**



Done!

Heapsort

It turns out that this method of constructing a binary heap is O(n)

See Cormen for details

This is efficient and requires no extra memory

We can use this for a worst case O(n log n) sorting algorithm for arrays which requires no auxiliary space (recall merge sort needed O(n) extra space):

Build binary heap in O(n) time

Smallest element is in correct place

Treat remaining array as a binary heap (second element will now be root) fix invariant using bubble down, second element now in correct place

Repeat until whole array is sorted

This is a kind of selection sort but is more efficient because we have some knowledge about what is beneath the root in the heap

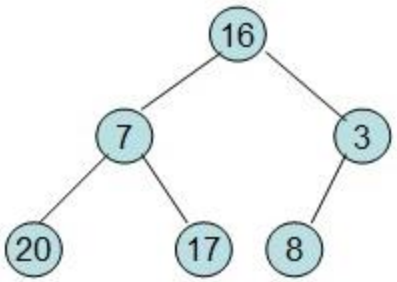
Heapsort

This is the ***heapsort*** algorithm

In practice, it is beaten by quicksort

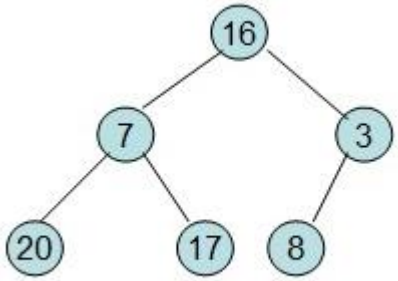
It is also not a stable sort

However, it gives us worst case O(n log n) and works in place so is useful in some situations

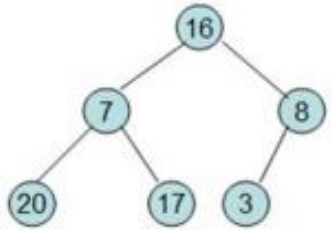


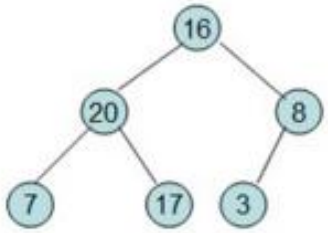
A heap can be classified further as either a "**max heap**" or a "**min heap**".

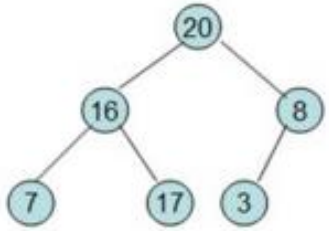
Heapsort

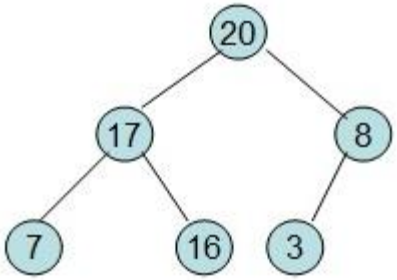


a[]={16,7,3,20,17,8}

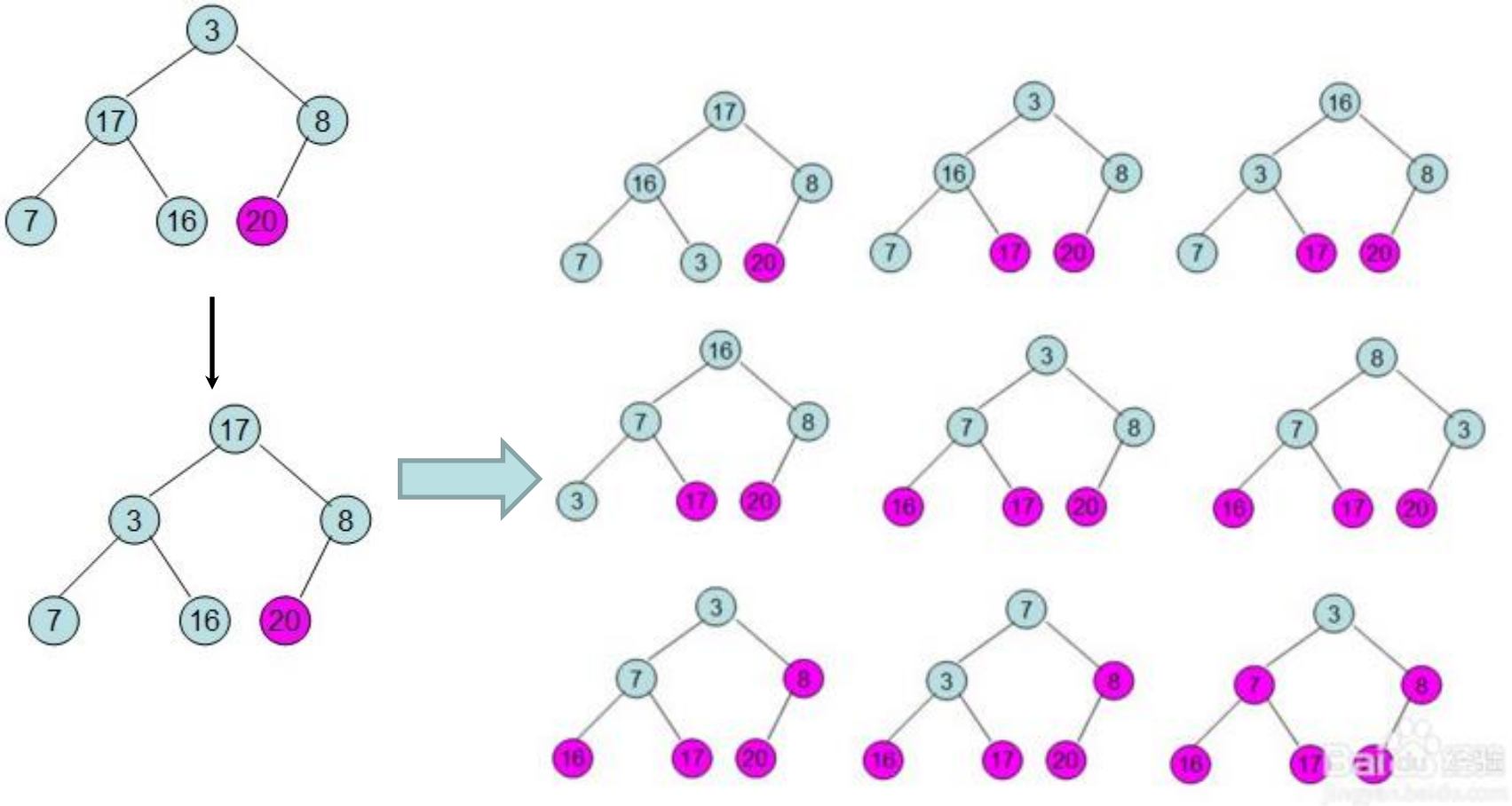








Heapsort


3
17
8
7
16
20
17
3
8
7
16
20
17
16
8
7
3
20
3
16
8
7
17
20
16
3
8
3
17
20
16
3
8
7
17
20
8
7
3
16
17
20
16
7
8
3
17
20
3
7
8
16
17
20
7
3
8
16
17
20
3
7
8
16
17
20

Conclusions

We should now know:

- What a priority queue is and the operations it supports
- Some applications of a priority queue
- Implementation using arrays or trees
- Implementation using a binary heap
- Heap sort

Next time:

- Introduction to graphs

Recommended reading for this lecture:

- Mehlhorn Chapter 6
- Cormen Chapter 6
- Skiena 3.5/3.6, 4.3